

项目技术架构说明

儿童成长观察台

技术架构与参考项目说明

Child Growth OS / Human Memory Orchestrator

基于当前代码、文档与本地参考项目快照生成

核心关键词

记忆编排

成长仿真

可观察

可回放

业务定位

用于儿童成长观察的数字人仿真与管理平台。

架构主线

React 控制台、后端接口、记忆编排与仿真引擎协同工作。

版本边界

使用虚构儿童世界，支持间接干预、审计与降级。

生成日期: 2026-06-03

资料范围: README、docs、backend/app、frontend/src、infra/docker-compose.yml、upstream_repos

1. 项目概览

本项目是一个用于儿童成长观察场景的数字人仿真与人格记忆平台。它把儿童成长世界、数字人记忆、半天事件回放、干预审计和成长报告放在一个可演示、可扩展的系统中。

业务价值

提供可解释的成长模拟平台：能创建虚构儿童世界，观察儿童与家庭、幼儿园、社区的互动变化。

核心能力

每次 step 代表半天，系统生成事件、更新状态指标、沉淀关系证据，并定期形成 7 天成长报告。

技术特点

前端负责观察和管理，后端统一掌控事实状态，记忆层负责长期上下文，仿真层负责事件推进。

安全边界

第一版只默认支持虚构儿童，不输出现实诊断建议，观察者只能间接干预，重要操作写入审计。

演示形态

当前 UI 已包含观察台、管理区、人格、记忆、文档、评测、服务状态等页面。

交付重点

先保障可观察、可回放、可解释，再逐步增强地图可视化、长期运行和运营控制能力。

2. 产品范围与边界

当前已具备的能力

- 儿童世界草稿：通过模板、年龄、照护者、同伴数和自然语言描述生成可审核草稿。
- 成长仿真：半天为单位推进，记录儿童 needs、发展域、关系边和事件时间线。
- 观察干预：观察者可注入环境事件、成人行为、NPC 行为、规则变化或 GM 事件。
- 回放报告：保存快照，预留分支预览，运行满 14 个半天 step 后生成 7 天报告。
- 管理后台：人格、对话、长期记忆、时间线、文档导入、评测和健康检查。

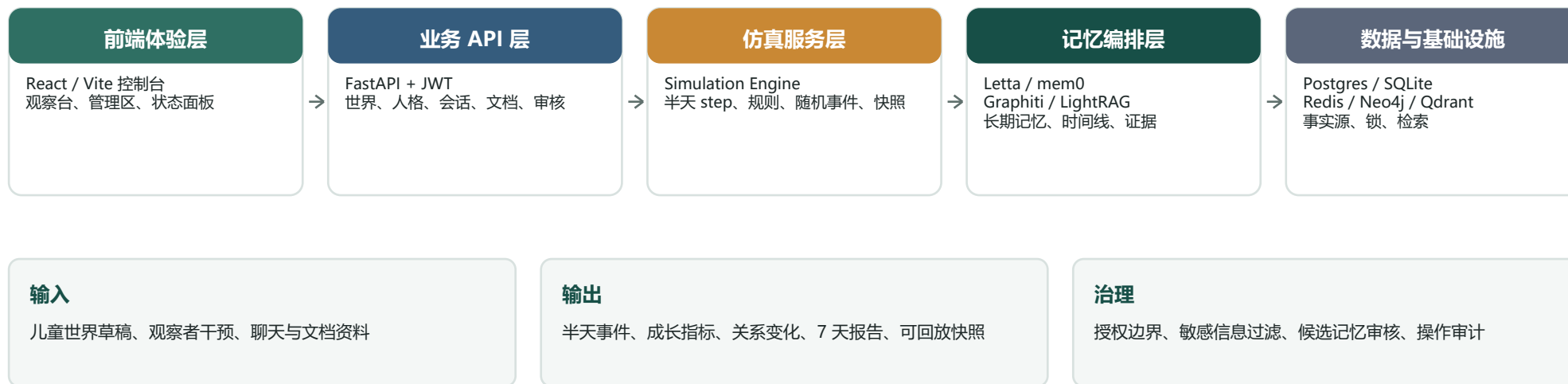
第一版明确不做

- 不做真实儿童身份建模，不采集照片、证件、病历或真实学校班级等可识别信息。
- 不提供医学、心理诊断或现实育儿建议，报告只解释虚构模拟中的观察线索。
- 不让前端或用户直接修改儿童核心状态，干预以环境输入进入后端规则结算。
- 不追求上百 agent 的大规模仿真，第一版适合 6 到 12 个数字人级别。
- 不直接复刻 AI Town 的 Convex 后端，避免引入第二套事实源和调度体系。

角色	系统中看到什么	系统内部支撑什么
业务负责人	可观察的成长模拟、干预记录、阶段报告	可复用场景、状态追踪、审计留痕
运营人员	世界列表、运行状态、事件时间线、服务健康	队列、快照、降级、可回放事件
内容/研究人员	儿童模板、场景、规则、发展域和关系指标	Persona、Memory、Rulebook、Growth Report

3. 总体技术架构

系统采用清晰分层：前端只负责交互和展示，后端掌控事实状态，仿真服务推进世界，记忆层提供长期上下文，基础设施提供持久化、检索和运行保障。



关键原则：让每一层承担清晰职责，避免前端、LLM、仿真引擎同时争夺同一份事实状态。

4. 前端体验层

观察台

- 儿童世界选择、刷新、单步推进、连续演进和运行状态控制。
- 家庭、幼儿园、社区/户外三类固定场景视图，展示儿童当前所在地。
- 儿童 needs、发展域、半天摘要、关系边、近期干预和事件时间线。
- 7 天报告、历史快照、分支预览占位，为后续对比演示预留入口。

技术栈	说明	业务价值
React + TypeScript + Vite	构建中文优先的管理控制台和观察台	交互快，易迭代，便于后续增加地图或可视化组件
lucide-react 图标	统一按钮和信息层级	界面更直观，减少说明文字依赖
浏览器 fetch API	统一调用 /api 下的后端接口	前后端职责清楚，便于部署和扩展

管理区

- 人格创建和初始化，支持通过描述生成虚构人格草稿与历史记忆。
- 对话控制台展示上下文证据，说明回答依据来自记忆、时间线或文档。
- 长期记忆、候选审核、时间线搜索、文档库导入与检索。
- 评测和服务状态页面，便于演示稳定性和依赖服务状态。

5. 后端与 API 层

后端是系统的主控层：负责鉴权、业务规则、数据落库、服务降级、仿真推进和对外 API。当前采用 FastAPI + SQLAlchemy async + Pydantic 的 Python 技术栈。

模块	主要接口或能力	业务作用
Auth	管理员登录、JWT token	控制后台访问，形成最小权限入口
Personas	人格创建、生成草稿、初始化 Letta agent	管理数字人的身份、边界和核心记忆
Conversations	会话、消息、上下文证据	支持可解释对话和记忆写入
Memory / Review	长期记忆、候选审核、核心记忆提案	避免敏感或低置信记忆直接污染人格
Documents	文档库、文本导入、检索	把日记、访谈、背景资料作为原文证据层
Worlds	儿童世界、地点、agent、规则、事件、干预、报告	驱动成长仿真和观察台核心体验
Health / Evals	服务状态、评测运行	为演示、验收和运维提供可观测入口

6. 记忆编排架构

项目的核心不是简单聊天，而是把不同类型的记忆放在不同层里管理：人格核心、长期偏好、时间事实、原文资料和冲突处理各司其职。

记忆层	负责什么	不负责什么	系统收益
Letta	有状态 agent、persona/human core memory、最终回答	大规模文档库、复杂时间线	让数字人保持一致人格和对话状态
mem0	长期事实、偏好、习惯、目标	完整对话和关系图谱	让稳定偏好可被长期记住
Graphiti	谁、何时、发生了什么、关系如何变化	全文检索	让事件可追溯，可按时间解释
LightRAG	日记、访谈、聊天归档、背景资料原文检索	每轮短期记忆	回答和报告可以回到原文证据
memory_orchestrator	检索路由、写入策略、冲突处理、上下文预算	让模型自由决定记忆治理	降低记忆混乱和成本浪费



7. 儿童成长仿真链路

儿童成长世界采用“草稿 - 审核 - 确认 - 半天推进 - 事件沉淀 - 报告输出”的闭环。每一步都强调可解释、可回放和安全边界。



对象	当前模型中如何体现	展示重点
儿童主角	SimAgent + AgentState + development metadata	needs、发展域、半天摘要、关键记忆
关系边	AgentRelationship	照护者、老师、同伴的熟悉度、温暖、信任安全等指标
成长报告	GrowthReport	主要经历、情绪模式、发展趋势、下周关注
干预	UserIntervention + AuditLog	只能注入环境或成人行为，不能直接改儿童状态

8. 数据与事实源

系统将不同数据放在不同事实源里，避免所有信息都堆在一个模型上下文中。核心业务状态在数据库，检索能力在图谱和向量服务，审计始终保留本地副本。

数据组	代表表或服务	职责
账号与审计	admin_users、audit_logs	后台访问、操作留痕和合规追踪
人格与会话	personas、persona_versions、conversations、messages	数字人身份、核心记忆、聊天记录
记忆与时间线	memory_records、events、graph_episode_links	长期记忆候选、事件时间线、Graphiti 关联
文档资料	corpora、documents、LightRAG	文本导入、原文检索、背景资料证据
仿真世界	simulation_worlds、world_locations、sim_agents、agent_states	世界时钟、场景、数字人和运行状态
仿真事件	simulation_actions、simulation_events、random_event_templates	行动、随机事件、GM 解释和状态变化
成长观察	child_world_drafts、agent_relationships、growth_reports、world_snapshots	儿童世界草稿、关系、成长报告和回放

9. 安全、隐私与合规边界

已体现在当前设计中的控制点

- 单管理员登录和 JWT 保护 API。
- 人格分为 fictional_persona 和 authorized_real_persona，真实人物必须确认授权。
- 儿童世界草稿带风险关键词检查，高风险可识别信息阻止确认创建。
- 观察者只能注入间接事件，不直接修改儿童状态。
- 规则修改、世界状态变化、草稿确认等关键动作写入审计日志。
- 外部服务不可用时保留本地降级路径，便于演示和故障隔离。

使用边界

- 这不是现实儿童诊断系统，而是虚构成长观察和产品验证平台。
- 不输入真实儿童姓名、照片、病历、证件、真实学校班级等可识别信息。
- 成长报告解释的是模拟世界的观察线索，不构成医学或心理建议。
- 进入真实业务试点前，需要补充多角色权限、数据脱敏、告知同意和留存策略。

风险	为什么需要控制	当前缓解方式
真实儿童信息误入	涉及隐私和合规风险	风险关键词、草稿审核、默认虚构儿童
记忆污染	错误记忆会影响后续行为	候选审核、置信度、来源和冲突提示
LLM 输出不可控	可能生成不稳定或高风险内容	结构化 schema、raw outcome 保留、needs_review 标记
状态被随意修改	成长轨迹失真	后端为事实源，干预只作为环境输入

10. 部署与运行架构

本地和演示环境通过 docker-compose 启动完整服务栈；开发模式也可分别启动 backend 和 frontend。正式部署时建议将数据库、向量库、图谱库和 LLM 服务配置外置。

服务	当前用途	部署建议
frontend	React 控制台，默认 8080	可静态部署或由 Nginx 托管
backend	FastAPI API，默认 8000	作为业务主控服务，需配置 CORS、JWT、数据库连接
worker	后台任务与仿真循环	和 backend 分开扩缩，避免接口请求被长任务阻塞
postgres	生产建议事实数据库	正式环境使用托管或独立实例，定期备份
redis	锁、队列、pub/sub	用于防止同一 world 并发 step
neo4j	Graphiti 图谱存储	用于时间事实和关系变化检索
qdrant	向量检索服务	用于 mem0 和文本检索的向量能力
letta / lightrag	agent runtime 与原文检索层	可独立部署，健康检查和降级策略要清晰

可观测

健康检查覆盖数据库、Letta、LightRAG、Neo4j、Qdrant 等关键依赖。

可降级

Letta、mem0、Graphiti、LightRAG 不可用时保留本地可审计路径。

可恢复

仿真按间隔保存 world snapshots，支持后续回放和恢复设计。

11. 参考项目与借鉴点

本项目保留多个开源项目作为本地快照或运行依赖来源。整体策略是借鉴成熟方向，但避免第一版复杂度失控。

参考项目	许可证/来源	本项目借鉴什么	第一版取舍
Concordia	Apache-2.0, 本地 upstream_repos/concordia	Game Master、环境裁判、agent/environment 组件	作为仿真裁判和后端 adapter 方向，不直接搬完整示例
AI Town	MIT, 本地 upstream_repos/ai-town	小镇地图、角色移动、对话气泡、实时交互体验	只做 UI 和产品形态参考，不引入 Convex 后端
Letta	官方服务运行，不 vendored	有状态 agent runtime 和核心记忆	用于人格状态和对话运行时
mem0	Apache-2.0, backend/vendor/mem0	长期事实、偏好、习惯和目标记忆	作为长期记忆层，并保留本地审计副本
Graphiti	Apache-2.0, backend/vendor/graphiti-core	带时间戳的事件图谱和关系变化	用于谁、何时、发生了什么的可追溯层
LightRAG	MIT, backend/vendor/lightrag-hku	原文资料、日记、访谈和背景文档检索	作为文档证据层，第一版只支持文本导入
RAGFlow	本地 upstream_repos/ragflow	复杂文档解析和知识库产品参考	暂不进入第一版，可作为后续 OCR/PDF/DOCX 能力参考

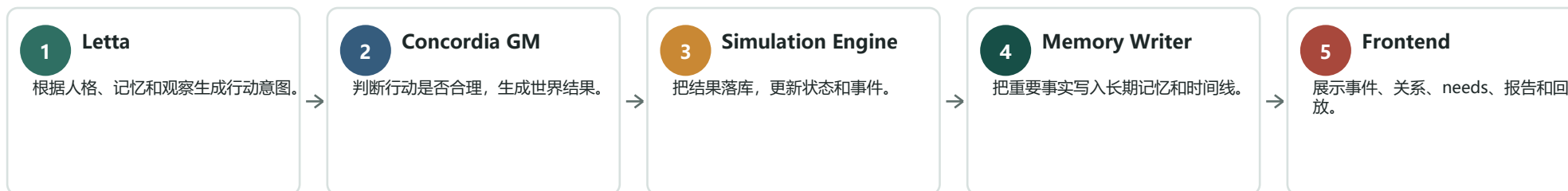
12. 与参考项目的关系

为什么不直接复制 AI Town

- AI Town 的主栈是 TypeScript + Convex + PixiJS；当前项目主控后端是 Python FastAPI。
- 直接并入会出现两套事实源、两套任务调度和两套事务模型。
- AI Town 适合参考地图、角色、对话、调试工具和产品体验，不适合作为第一版后端基座。
- 如复用 assets，需要单独做素材许可证审计。

为什么选择 Concordia 做裁判参考

- Concordia 与当前后端同为 Python 生态，更适合嵌入 worker 或 adapter。
- 它适合负责“世界是否允许以及发生了什么”，让行动结果有统一裁判。
- Letta 继续负责“这个人想做什么”和人格上下文，避免两套 agent 同时做最终裁决。
- 当前实现保留 deterministic fallback，确保没有 LLM 或外部服务时仍能演示。



13. 阶段路线图

当前项目已经超过纯概念阶段，具备后端模型、API、前端观察台和儿童成长专项逻辑。后续建议按可演示能力逐步推进，而不是一次性追求完整游戏化仿真。

阶段	目标	交付物
当前状态	完成记忆编排原型、儿童成长世界、半天推进、观察台和管理区	可本地启动和演示的 MVP 基础
短期 1	强化演示闭环	默认样例世界、稳定演示脚本、关键事件模板、报告样式优化
短期 2	增强可视化	地图视图、角色位置动画、事件浮层、WebSocket/SSE 状态流
中期 1	长期运行	Redis 锁硬化、worker 任务队列、快照恢复、成本和速率限制
中期 2	产品化治理	多角色权限、数据脱敏、导出报告、操作审计查询、试点评估体系
后续扩展	更复杂资料和场景	PDF/DOCX/OCR 导入、更多成长模板、分支仿真、定制场景

第一演示目标

创建小雨的成长世界，推进 3 到 5 个半天，展示状态指标、关系和事件变化。

第一验收目标

运行满 14 个半天 step，自动生成 7 天报告，并能查看历史快照。

第一稳定目标

外部服务不可用时仍能通过 fallback 产生可解释事件。

14. 系统演示流程

推荐讲解顺序

- 先讲业务场景：这是儿童成长观察的虚构模拟平台，不是诊断系统。
- 再讲产品体验：观察台如何创建世界、推进半天、查看变化和生成报告。
- 然后讲架构：前端展示、后端主控、仿真推进、记忆分层、数据持久化。
- 最后讲参考项目：Concordia 做仿真裁判参考，AI Town 做小镇交互参考。

演示脚本

- 使用默认表单生成“好奇外向型”儿童世界草稿。
- 审核草稿并确认创建，说明风险标记和安全边界。
- 推进半天，展示场景、needs、发展域、事件时间线。
- 注入一次间接干预，例如老师入园时温和提醒，再推进半天查看影响。
- 展示管理区：人格、记忆、文档、评测、服务状态。

常见问题	说明
这个系统是否会保存真实儿童信息？	第一版不建议也不需要保存真实儿童信息，默认创建的是虚构儿童世界，并带风险检查。
AI 会不会直接决定儿童状态？	不会。状态由后端规则和仿真流程结算，LLM/GM 只提供解释和候选结果，异常会标记审核。
为什么要有这么多记忆组件？	不同信息有不同生命周期：偏好、时间事件、文档原文和人格核心需要分层管理，才能可解释和可控。
后续能否做更真实的地图和动画？	可以。当前后端已经提供世界状态和事件接口，前端可逐步升级为地图、角色和实时状态流。

15. 资料来源与版本说明

本 PDF 基于当前工作区文件生成，内容偏架构展示和产品说明，不替代代码审计、信息安全测评或正式验收文档。

资料	用途
README.md	项目定位、启动方式、第一轮验证、重要边界
docs/ARCHITECTURE.md	记忆分层、读写路径、冲突处理、降级策略
docs/SOCIAL_SIMULATION_PLAN.md	社会仿真目标架构、参考项目、路线图和风险
docs/THIRD_PARTY_NOTICES.md	第三方项目来源、许可证和本地 vendoring 说明
docs/RUNBOOK.md	健康检查、仿真 smoke test 和常见问题
backend/app	FastAPI、模型、路由、仿真引擎、儿童成长逻辑和服务集成
frontend/src	观察台、管理区、浏览器 API 调用层和页面能力
infra/docker-compose.yml	本地运行栈和服务依赖
upstream_repos	Concordia、AI Town、mem0、Graphiti、LightRAG、Letta、RAGFlow 等参考快照

生成路径：output/pdf/child_growth_os_architecture_reference_report.pdf